



Presentation By:

RAJASHEKAR G.S
Java Full Stack Trainer

Object Oriented Concepts and Java Implementation - 3

Session 5

Review

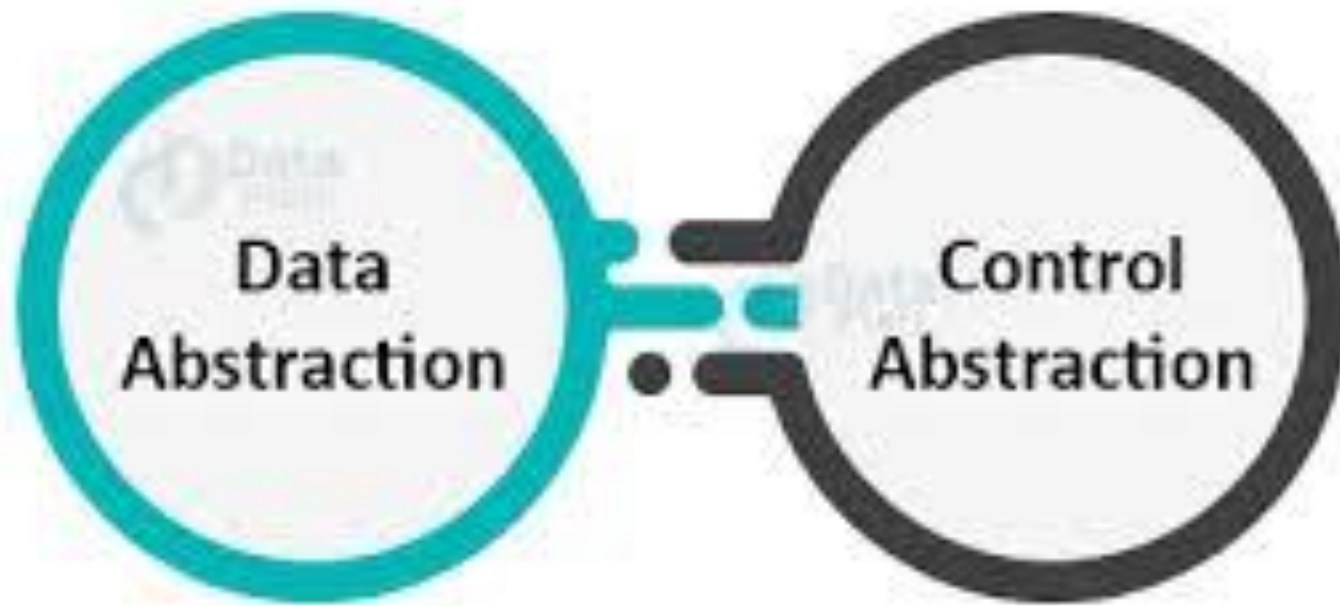
- ❑ Inheritance allows the creation of hierarchical classifications.
- ❑ Inheritance allows the reusability of code.
- ❑ All methods and properties of the base class are inherited by objects of the derived class except the constructors.
- ❑ Polymorphism is the ability of different objects to respond to the same message in different ways.
- ❑ Overloaded methods are examples of static polymorphism and overridden methods are examples of dynamic polymorphism.

Objectives

- **Define Abstraction .**
- **Explain Abstract Methods & Classes.**
- **Define Interfaces**
- **Implement Multiple Inheritance using Interfaces.**



Types of Abstraction in Java



What is Java Abstraction?

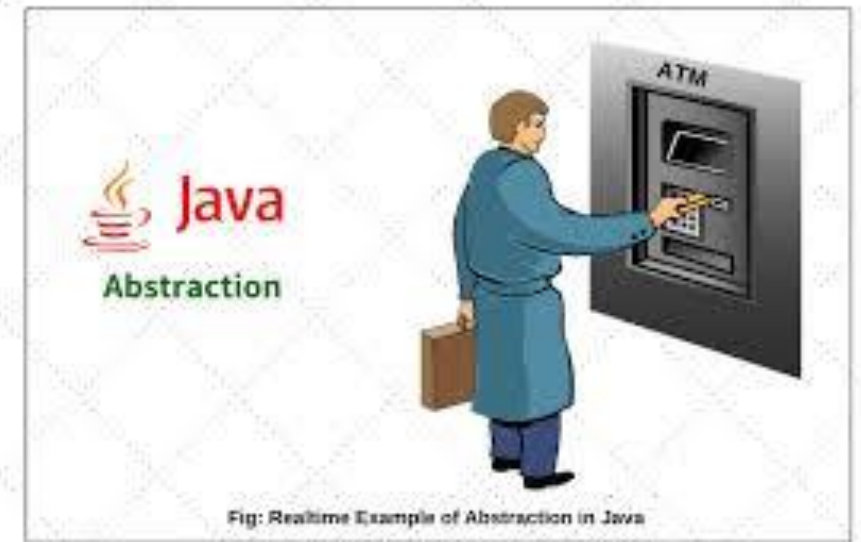
- Abstraction is nothing but the quality of dealing with ideas rather than events.
- It basically deals with hiding the internal details and showing the essential things to the user.



- Here, you don't know the internal processing of how a call is generated, that's the beauty of abstraction.

What is Abstraction ?

- Abstraction is a fundamental concept in object-oriented programming (OOP) in Java.
- It is a mechanism that allows you to show only the necessary information to the outside world while hiding the internal details.
- In Java, abstraction is achieved through the use of abstract classes and interfaces.
- Abstraction helps in reducing complexity and improving code organization by providing a high-level view of the program without revealing the internal implementation details.
- It enables developers to focus on the essential functionality while hiding the implementation details.



Realtime Examples of Abstraction in Java



We all use an **ATM machine** for cash withdrawal, money transfer, retrieve min-statement, etc. in our daily life.

But we don't know internally what things are happening inside ATM machine when you insert an ATM card for performing any kind of operation.



A **car** owner knows how to drive it. He knows about various components of car and how to use them.

But he need know how they function.



When you need to send **SMS** from your mobile, you only type the text and send the message.

But you don't know the internal processing of the message delivery.



We login to your **Amazon** account online, you enter your user_id and password and press login,

What happens when you press login, how the input data sent to amazon server, how it gets verified is all abstracted away from the you.

Abstraction



Abstraction is a process of hiding the implementation details and showing only functionality to the user.



It shows only essential things to the user and hides the internal details.



Ex: sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.



There are two ways to achieve abstraction in java

Abstract class (0 to 100%)

Interface (100%)

Abstract keyword



abstract is a non-access modifier in java



It is applicable for classes, methods but **not** variables.



It is used to achieve abstraction which is one of the pillar of Object Oriented Programming(OOP).



The abstract modifier indicates that the thing being modified has a missing or incomplete implementation.

abstract Modifier 3-1



Certain methods in the superclass do not contain any logic and need to be overridden in the subclass.



The keyword `abstract` is used for such methods.



The subclass provides implementation details of such abstract methods.



The syntax is: **`abstract type method_name(parameter_list);`**

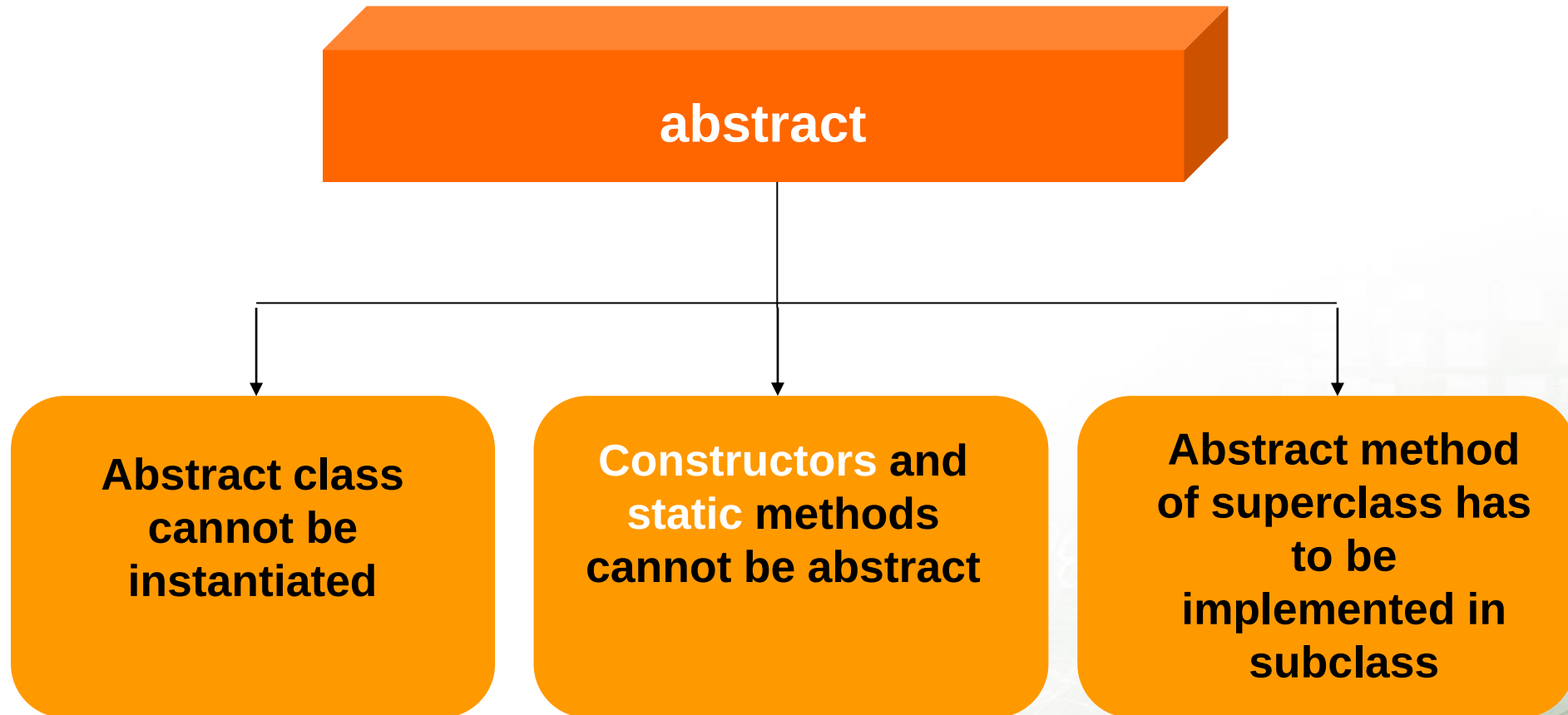


Any class that contains one or more abstract methods must be declared as `abstract`.



The class keyword should be preceded by the `abstract` keyword when declaring an abstract class.

abstract Modifier 3-2



- ❑ An abstract class can have object references that can point to a subclass object.

Abstract class & Method in Java

- ❑ Abstract class is a restricted class that cannot be used to create Objects.
- ❑ A class which is declared as abstract is known as an abstract class.
- ❑ It can have abstract and non-abstract methods.
- ❑ It needs to be extended and its method implemented.
- ❑ It cannot be instantiated.
- ❑ A method which is declared as abstract and does not have implementation is known as an abstract method.

```
abstract class Sports { // abstract class sports  
    abstract void jump(); // abstract method  
}
```

- You can achieve 0-100% abstraction using an abstract class.

Rules for Java Abstract class



1

An abstract class must be declared with an abstract keyword.

2

It can have abstract and non-abstract methods.

3

It cannot be instantiated.

4

It can have final methods

5

It can have constructors and static methods also.

Abstract class Usage

- ❑ Abstract class is used when you know partial implementation:
 - where say out of 5 methods,
 - you know implementation of 3 methods
 - and don't know implementation of 2 methods.
- ❑ In that case 2 methods will be abstract and you need to rely on implementer as a contract to must provide body of abstract methods to accomplish the task.



When to use Abstract class



- Consider we want to start a service like **Bulk SMS sender**, where we take orders from various telecom vendors like Airtel, France Telecom, Vodafone etc.
- For this, we don't have to setup our own infrastructure for sending SMS like Mobile towers.
- But we need to take care of government rules:
 - Like after 9PM, we should not send promotional SMS,
 - We should also not send SMS to users registered under Do Not Disturb(DND) service etc.
 - Remember, we need to take care of government rules for all the countries where we are sending SMS.
- For infrastructure like towers, we will be relying on vendor who is going to give us order.
Eg: Vodafone request us for bulk messaging, in that case we will use Vodafone towers to send SMS.



Bulk SMS Sender Template

```
public void establishConnectionWithYourTower(){  
    //connect using vendor way.  
    //we don't know how, candidate for abstract method  
}  
public void sendSMS(){  
    establishConnectionWithYourTower();  
    checkForDND();  
    checkForTelecomRules();  
    //sending SMS to numbers...numbers.  
    destroyConnectionWithYourTower()  
}  
public void destroyConnectionWithYourTower(){  
    //disconnect using vendor way.  
    //we don't know how, candidate for abstract method  
}  
public void checkForDND(){  
    //check for number present in DND.  
}  
public void checkForTelecomRules(){  
    //Check for telecom rules.  
}
```

Bulk SMS Sender Template



- Methods we know is "sendSMS()", "checkForDND()", "checkForTelecomRules()".
- Methods we don't know is "establishConnectionWithYourTower()", "destroyConnectionWithYourTower()".
- In this case, Abstract class will be helpful, because you know partial things like "checkForDND()", "checkForTelecomRules()" for sending sms to users.
- But we don't know how to `establishConnectionWithTower()` and `destroyConnectionWithTower()` and need to depend on vendor specific way to connect and destroy connection from their towers.

Bulk SMS Sender Template

```
abstract class SMSSender{

    abstract public void establishConnectionWithYourTower();

    public void sendSMS(){
        /*establishConnectionWithYourTower();
        checkForDND();
        checkForTelecomRules();

        sending SMS to numbers...numbers.*//
    }

    abstract public void destroyConnectionWithYourTower();

    public void checkForDND(){
        //check for number present in DND.
    }
    public void checkForTelecomRules(){
        //Check for telecom rules }
    }
```

Bulk SMS Sender Template

```
class Vodafone extends SMSSender{

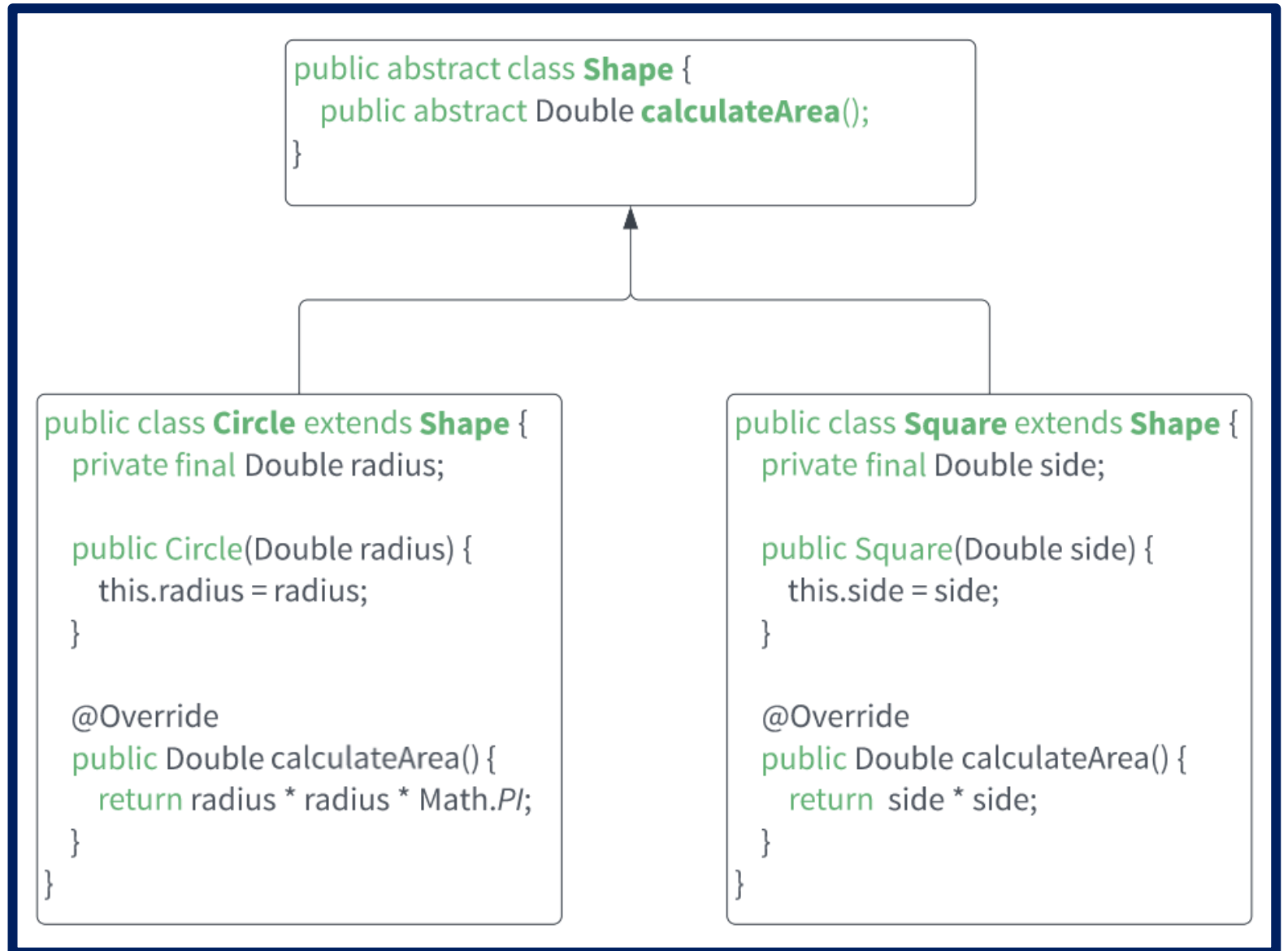
    public void establishConnectionWithYourTower() {
        //connecting using Vodafone way
    }
    public void destroyConnectionWithYourTower() {
        //destroying connection using Vodafone way
    }
}

class Airtel extends SMSSender{

    @Override
    public void establishConnectionWithYourTower() {
        //connecting using Airtel way
    }

    @Override
    public void destroyConnectionWithYourTower() {
        //destroying connection using Airtel way
    }
}
```

Abstract Class & Methods Demo



Abstraction (In Java)

```
public abstract class Electronic
{
    boolean powerOn = false;

    public boolean isOn()
    {
        return powerOn;
    }

    public void turnOn()
    {
        powerOn = true;
        System.out.println("Turned on");
    }

    public void turnOff()
    {
        powerOn = false;
        System.out.println("Turned off");
    }

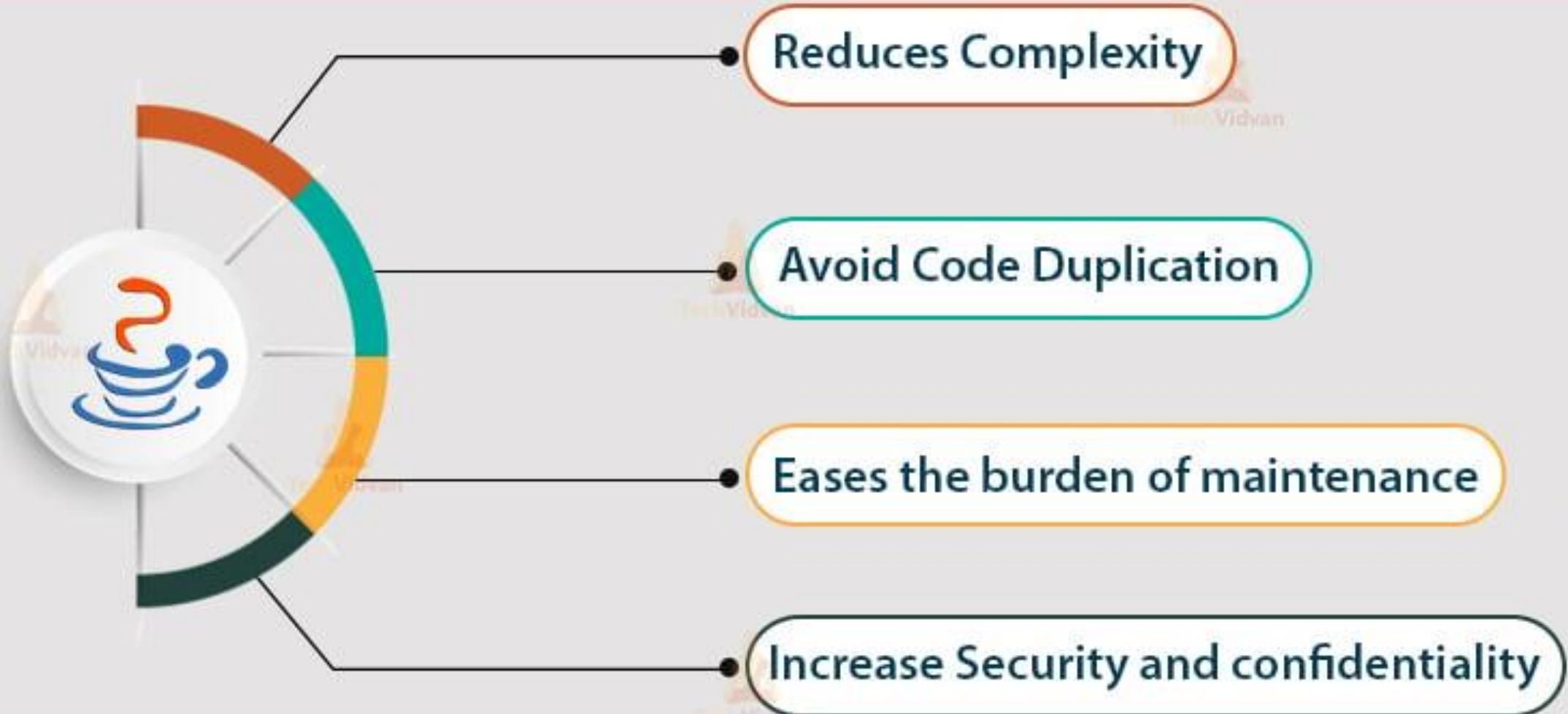
    public abstract void operate();
}
```

```
public class Toaster extends Electronic
{
    ...
    public void operate()
    {
        /*Toaster's own implementation of
        operate()
        */
    }
}

public class IPod extends Electronic
{
    public void operate()
    {
        /* IPod's own implementation
        of operate()
        */
    }
}
```

Oops!

Advantages of Java Abstraction



Interfaces in Java

Interfaces in Java

Interface is a blueprint of a class that defines a contract.

It specifies a set of methods that must be implemented by any class that implements it.

An interface cannot be instantiated on its own and cannot contain any implementations for its methods.

It is used to achieve abstraction, modularity, and single inheritance.

Interfaces are often used to define a common set of methods that multiple classes can implement to achieve a common goal.

In other words, an interface is a way to define a contract that must be followed by any class that implements it.

Interfaces

An interface in the Java programming language is an abstract type that is used to specify a behavior that classes must implement.

- Interface is a Java Element that contains
- Contains Collection of methods with no code segments.
- Can have only public members.
- Can contain constant variables which are public, static and final by default.
- Can extend other interface using extends keyword.
- Cannot contain instances of itself.

Interfaces

- Java Interface also represents the IS-A relationship.
- It cannot be instantiated just like the abstract class.
- Since Java 8, we can have default and static methods in an interface.
- Since Java 9, we can have private methods in an interface.
- The interface in Java is a mechanism to achieve abstraction.

Why use Java interface?

It is used to achieve abstraction.

1

2

By interface, we can support the functionality of multiple inheritance.

It can be used to achieve loose coupling.

3

Loose coupling means reducing dependencies of a class that use different class directly.

Tight coupling means classes and objects are dependent on one another.

Why Interfaces ?

Inheritance facilitates overriding of methods from Single Class.

Interface allows overriding of methods from multiple classes.

In Inheritance class may extend only one class.

For reusing functionality from multiple sources, **Java provides Multiple Inheritance through Interfaces.**

Interfaces reduces the code to greater extent.

Syntax – Interface Declaration

Can extend
another interface

Declared with the
interface keyword

```
public interface Dogs extends Animals {  
  
    public static final int genus = 25;  
    public abstract void sizeIt();  
    public void feedIt();  
  
}
```

A collection of methods
with no code statements

Can contain constants:
public,
static, and final

Can have only public
members

Class v/s Interface

- An interface is a collection of abstract methods.
- A class implements an interface, thereby inheriting the abstract methods of the interface.
- An interface is not a class.
- Writing an interface is similar to writing a class, but they are two different concepts.
- A class describes the attributes and behaviors of an object.
- An interface contains behaviors that a class implements.
- An interface is not extended by a class; it is implemented by a class.

Difference between Class & Interface in Java

Class

Class can instantiate variable & create object

Class can contain concrete methods

The access specifiers used with classes are private, protected and public



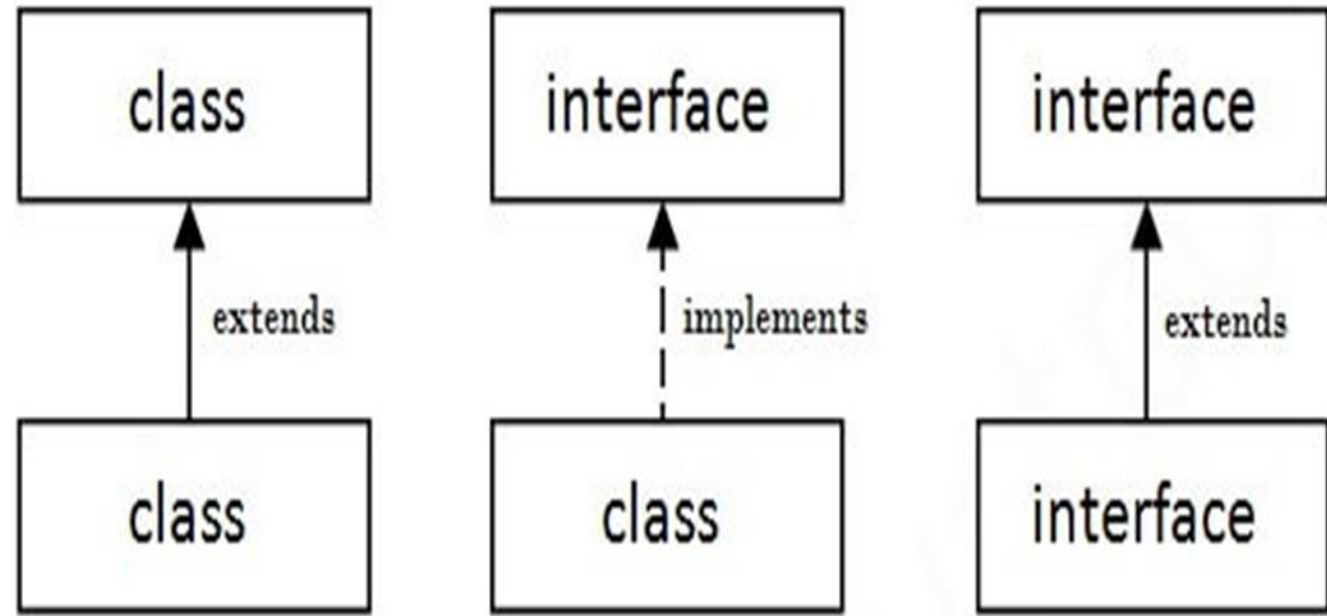
Interface

Interface can't instantiate variable & create an object

The interface can't contain concrete methods

In interface only one public specifier is used

The Relationship between classes and interfaces



As shown in the figure a class extends another class, an interface extends another interface, but a class implements an interface.

Implementing Interfaces

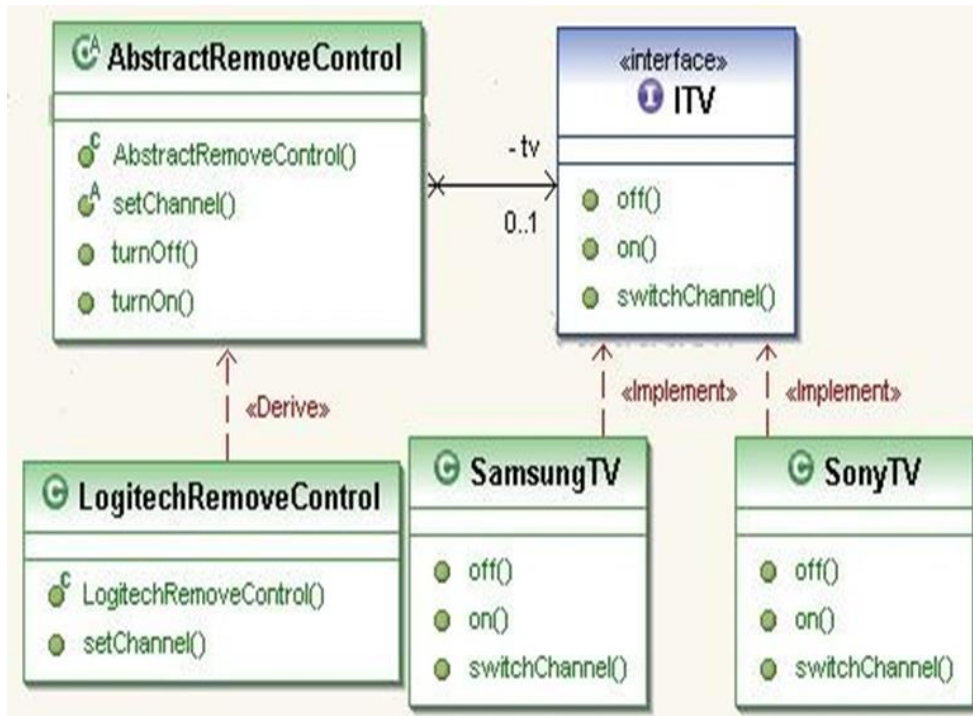
- Class implements an interface :
 - We can think of the class as signing a contract, agreeing to perform the specific behaviors of the interface.
- If a class does not perform all the behaviors of the interface, the class must declare itself as abstract.
- A class uses the **implements** keyword to implement an interface.
- The implements keyword appears in the class declaration following the extends portion of the declaration.

```
interface Vehicle{
    void type();
}

class Car implements Vehicle{
    public void type() {
        System.out.println("Car");
    }

    public static void main(String args[]){
        Vehicle vehicle = new Car();
        vehicle.type();
    }
}
```

Television Interface & Implementation



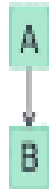
```
public interface ITV {
    public void on();
    public void off();
    public void switchChannel(int channel);
}
```

```
public class SamsungTV implements ITV {
    public void on() {
        System.out.println("Samsung is turned on.");
    }

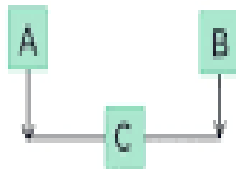
    public void off() {
        System.out.println("Samsung is turned off.");
    }

    public void switchChannel(int channel) {
        System.out.println("Samsung: channel -" + channel);
    }
}
```

Single inheritance



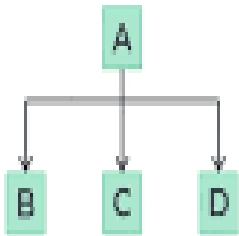
Multiple inheritance



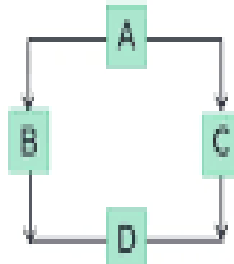
Multilevel inheritance



Hierarchical inheritance



Hybrid inheritance



TutorialsPoint

Types of Java Inheritance



01 *Single Inheritance*

02 *Multilevel Inheritance*

03 *Hierarchical Inheritance*

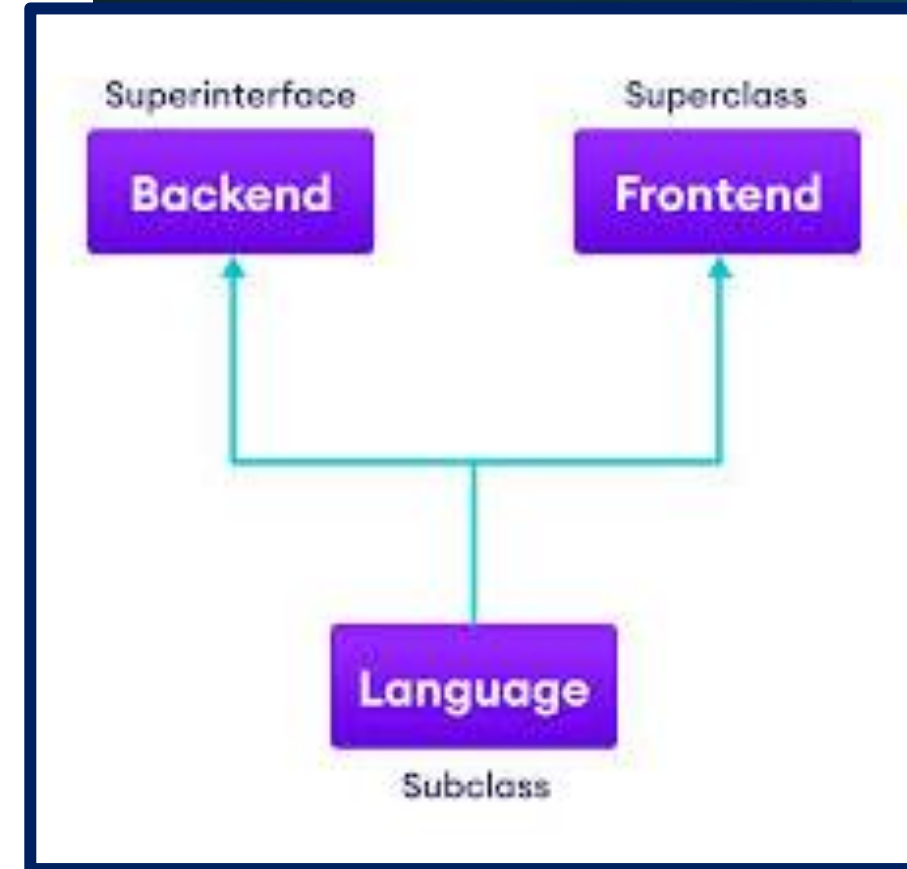
04 *Multiple Inheritance*

05 *Hybrid Inheritance*

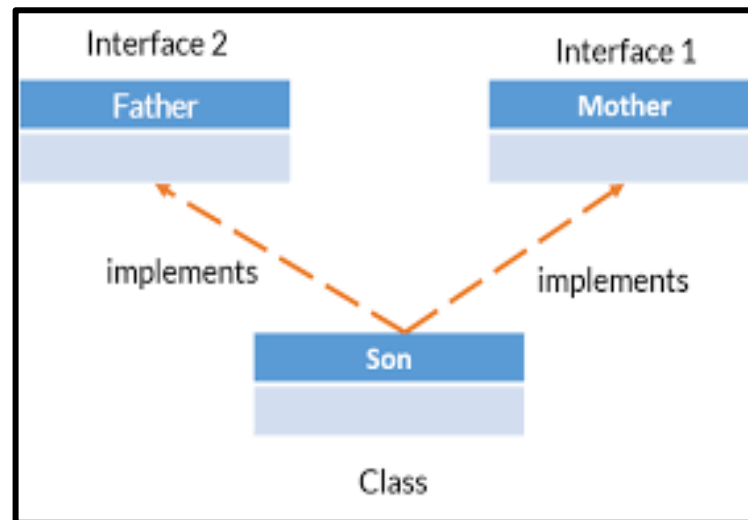
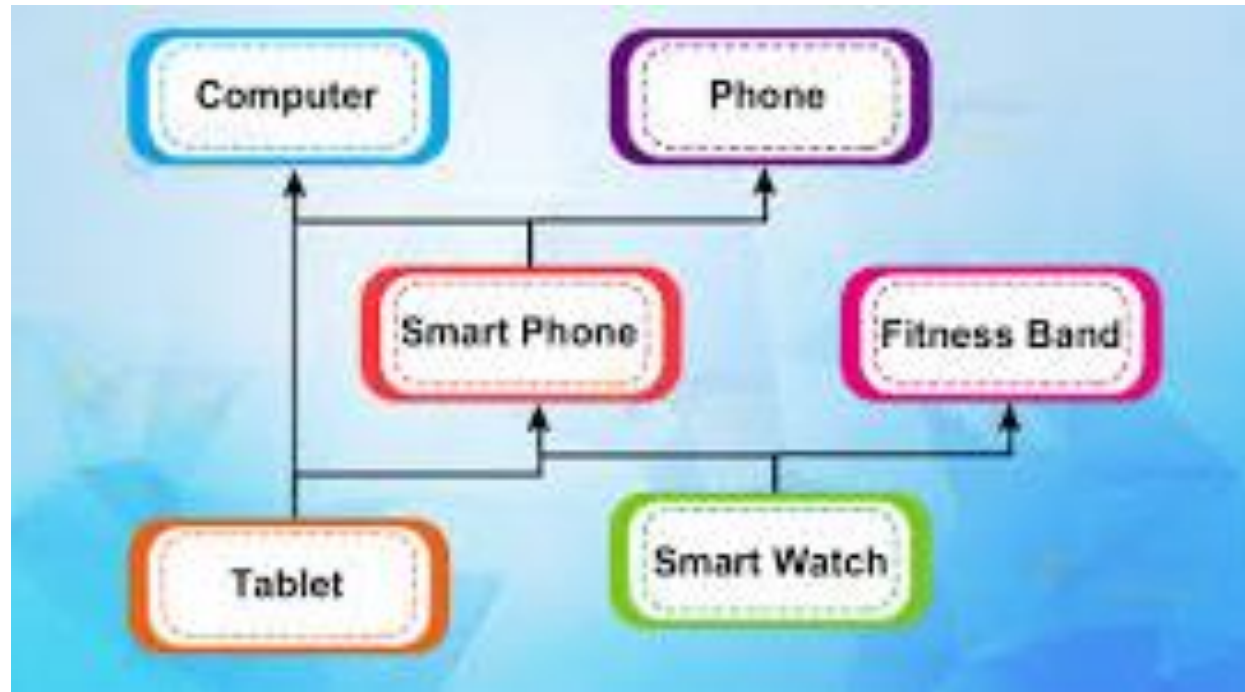
Tech Vidvan

Multiple inheritance in Java by Interface

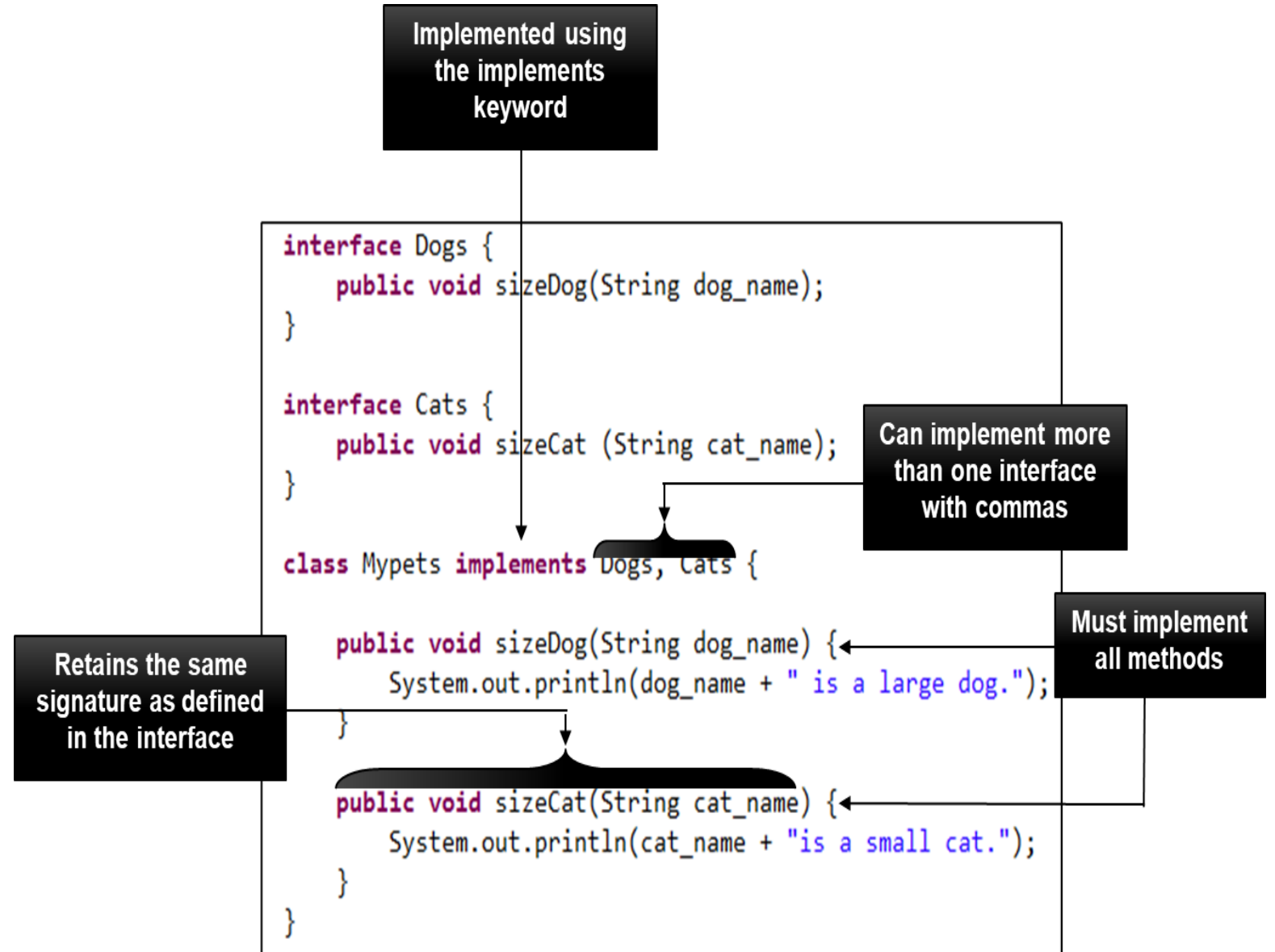
- Multiple inheritance in Java is a feature that allows class to inherit properties and behavior from more than one superclass.
- However, Java does not support multiple inheritance of classes directly.
- Instead, it provides the concept of interfaces, which can be used to achieve multiple inheritance.
- A class can implement multiple interfaces, and interfaces can extend multiple interfaces.
- This allows for a form of multiple inheritance, where a class can inherit behavior from multiple interfaces.



Multiple Inheritance in Java Examples



Interface Implementation Demo



Multiple Inheritance with Interface & Class

```
interface Backend {  
  
    // abstract class  
    public void connectServer();  
}  
  
class Frontend {  
  
    public void responsive(String str) {  
        System.out.println(str + " can also be used as frontend.");  
    }  
}
```

```
// Language extends Frontend class  
// Language implements Backend interface  
class Language extends Frontend implements Backend {  
  
    String language = "Java";  
  
    // implement method of interface  
    public void connectServer() {  
        System.out.println(language + " can be used as backend language.");  
    }  
}
```

```
public static void main(String[] args) {  
  
    // create object of Language class  
    Language java = new Language();  
  
    java.connectServer();  
  
    // call the inherited method of Frontend class  
    java.responsive(java.language);  
}  
}
```

Multiple Inheritance with Interfaces

```
interface Message1 { // interface 1
    public void method();
}
interface Message2 { // interface 2
    public void method(int id);
}
public class IntrfExample3 implements Message1, Message2 {
    // using the method here with different definition
    public void method() {
        System.out.println("Tutorialspoint");
    }
    public void method(int id) {
        System.out.println("ID: " + id);
    }
    public static void main(String []args){
        // object creation
        IntrfExample3 exp = new IntrfExample3();
        // method calling
        exp.method();
        exp.method(125);
    }
}
```

When to use Interface ?

- Consider we want to start a service like "[makemytrip.com](#)" or "[expedia.com](#)".
- We are responsible for displaying the flights from various flight service company and place an order from customer.

Let's keep our service as simple as,

- Displaying flights available from vendors like "[airasia](#)", "[british airways](#)" and "[emirates](#)".
- Place and order for seat to respective vendor.
- In this application, we don't own any flight.
- We are just a middle man/aggregator and our task is to first enquire "[airasia](#)", then enquire "[british airways](#)" and at last enquire "[emirates](#)" about the list of flights available.
- If customer opts for booking then inform the respective flight vendor to do booking.



Flight Service Company Demo

- This means we only care for method `"getAllAvailableFlights()"`
- `"getAllAvailableFlights()"` from "airasia" may have used SOAP service to return list of flights.
- `"getAllAvailableFlights()"` from "british airways" may have used REST service to return list of flights.
- `"getAllAvailableFlights()"` from "emirates" may have used CORBA service to return list of flights.
- But we don't care how it is internally implemented and what we care is the contract method `"getAllAvailableFlights"` that all the flight vendor should provide and return list of flights.
- Similarly, for booking I only care for method `"booking()"` that all vendors should have, internally how this vendors are doing booking that I don't care.
- So we can say that we know the contract that irrespective of who the Flight vendor is, we need `"getAllAvailableFlights()"` and `"booking()"` method from them to run our aggregator service.

Solution for Flight Service Company Demo

```
interface FlightOpeartions
{
    void getAllAvailableFlights();
    void booking(BookingObject bookingObj);
}
```

```
class BritishAirways implements FlightOpeartions {

    public void getAllAvailableFlights(){
        //get british airways flights in the way
        //they told us to fetch flight details.
    }

    public void booking(BookingObject flightDetails){
        //place booking order in a way British airways
        //told us to place order for seat.
    }

}
```



Abstract Class vs Interface

- Abstract class is used when you know something and rely on others for what you don't know.
- Here it is partial abstraction as some of the things you know and some you don't know.
- Interface is used when you want to define a contract and you don't know anything about implementation.
- Here it is total abstraction as you don't know anything.

Interface

I only know method names that
I will require for my job to be done.
You have to provide body for those
methods.

Interface

Implementer

contract



Abstract class

abstract class

Some methods I know.

Some methods I don't know and I will
depend upon you to provide.

Implementer 2



Some methods I know.

Some methods I don't know and I will
depend upon you to provide.

Implementer 1



Difference between Abstract class and interface

Abstract class and interface both are used to achieve Abstraction where we can declare the abstract methods.

Abstract class	Interface
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods.
2) Abstract class doesn't support multiple inheritance .	Interface supports multiple inheritance .
3) Abstract class can have final, non-final, static and non-static variables .	Interface has only static and final variables .
4) Abstract class can have static methods, main method and constructor .	Interface can't have static methods, main method or constructor .
5) Abstract class can provide the implementation of interface .	Interface can't provide the implementation of abstract class .

When to Use Abstract classes / Interface?



Interface and Abstract Class are the two different ways to achieve Abstraction in OOP Languages.



Interface provides 100% abstraction, i.e all methods are abstract.



Abstract class provides 0 to 100% abstraction, i.e it may have or may not have abstract methods.



We can use Interface when we want all the functionality of a type to be implemented by the client.



We can use Abstract Class when some common functionality can be provided by Abstract Class implementer and client will be given chance to implement what he needs actually.

Interface vs Abstract Classes



- Interface is used when you want to define a contract and you don't know anything about Implementation. – Total Abstraction



Eg: makemytrip,expedia



- Abstract class is used when you know something and rely on others for what you don't know. – Partial Abstraction



- Eg: Bulk Sms Sender

Summary

- ❑ Abstraction in Java is a fundamental concept in object-oriented programming.
- ❑ Abstraction is a process of hiding the implementation details and showing only functionality to the user.
- ❑ In Java, abstraction is achieved through abstract classes, interfaces, and abstract methods.
- ❑ Abstract classes and interfaces cannot be instantiated; they serve as blueprints for other classes to follow.
- ❑ Abstract methods, on the other hand, are declared without implementation and must be implemented by any subclass that inherits from the abstract class.
- ❑ Abstraction helps in reducing complexity, improving code reusability, and enhancing modularity.

Summary

- ❑ Interfaces in Java are abstract classes that define a contract that must be implemented by any class that implements it.
- ❑ An interface cannot be instantiated on its own and is used to provide a blueprint for the implementation of methods and properties by its implementing classes.
- ❑ Interfaces are often used to define the behavior of a group of classes that share a common purpose or function, such as network protocols or file formats.
- ❑ They are a fundamental concept in object-oriented programming and are used extensively in Java programming.

An aerial photograph of a multi-lane highway bridge spanning a body of turquoise water. A large, white, brushstroke-style graphic is centered over the bridge, containing the text "Thank You".

Thank You